

Official Duria AI Documentation & Architecture

This document serves as the master blueprint for integrating "DURIA" (the overarching AI Assistant) into the Durio application. It outlines the architecture, implementation strategy, challenges, and cost-saving methodologies.

1. Vision & Constraints

- **Multi-User Architecture:** Durio is a multi-user platform. DURIA must strictly operate within the boundaries of the authenticated user's data, triggering only server actions protected by `getUserId()`.
- **Fully Free Operation:** Development and initial deployment must incur zero cost.
- **Token-Friendly Strategy:** To prevent excessive token usage and API costs, DURIA will not blindly search the entire database. Instead, it relies on a **Semi-Automated Workflow** (detailed below).

2. AI Model Selection

To maintain the "fully free" constraint, DURIA will utilize one of the following models:

- **Google Gemini (Free Tier):** Highly recommended. Fast, handles large contexts, and natively supports structured JSON output (Function Calling) which is vital for backend mutations.
- **Groq API:** Offers lightning-fast Llama-3 models with a generous free tier for developers.
- **Local LLMs (Ollama):** 100% free and private execution, though hardware dependent.

3. Global Module Registry

To ensure the AI understands its environment, the app is controlled by a central `APP_REGISTRY` (located in `src/config/modules.ts`).

- **Purpose:** Acts as the absolute source of truth. It dictates which UI elements render, what modules are enabled by the user or system, and what features DURIA is allowed to interact with.
- **Structure:** Contains `GLOBAL` (fancy mode), `LAYOUT` (header, sidebar), and `MODULES` (all current and future app features).

4. The Token-Friendly Strategy (Semi-Auto Workflow)

Passing a user's entire database to the AI for every prompt will hit rate limits, increase latency, and eventually cost money. To solve this, DURIA uses a **Semi-Auto Workflow**:

1. **Manual Context Injection:** If the AI needs to look at notes or tasks to answer a prompt, the user manually selects references via a "Context Selector" UI (e.g., selecting specific notes, or setting a filter like "Pending tasks for this week").
2. **Local Caching:** The app fetches this specific data and stores it locally as a "Context Payload".
3. **Reusability:** When the user asks a follow-up question, the system will explicitly ask: *"Use the same cached data, or renew with new filters?"*
4. **Benefit:** This guarantees we only send the exact tokens required to the AI, reducing lag dramatically and entirely preventing AI hallucinations.

5. Implementation Phases

- **Phase 1: Foundation (Completed)** - Establish the `APP_REGISTRY` and link the UI components (Sidebar/Nav) to it. Create the Feature Guide.
- **Phase 2: User Settings & Fancy Mode** - Build the UI in the Settings module allowing users to toggle individual modules on/off and enable/disable "Fancy Mode" to maximize UI performance.
- **Phase 3: Semi-Auto Context Engine** - Build the UI for users to manually select and cache data (Notes, Todos) to be used as AI context.
- **Phase 4: Read-Only Assistant** - Integrate the LLM API. The AI receives the prompt + the cached context + the Feature Guide to accurately answer user queries.
- **Phase 5: Action-Taker AI** - Enable Function Calling via strict Zod schemas so the AI can execute backend commands (e.g., `createTodo` , `createEvent`) based on natural language.

6. Challenges & Solutions

Challenge	Proposed Solution
Token Cost / Latency	Rely on the Semi-Auto Workflow (curated local context) instead of blind background fetching.
Hallucinated Actions	Strict Zod Validation: DURIA's function calls must strictly match our frontend form validation schemas before being executed.
Data Leakage (Multi-user)	Backend Isolation: DURIA only executes Next.js Server Actions, which inherently authenticate via <code>getUserId()</code> .
Context Window Limits	If a user attaches massive notes to the payload, the client may need to chunk or summarize the data locally before transmission.

